

LAPORAN PRAKTIKUM PRAKTIKUM STRUKTUR DATA

Disusun oleh

Nama : Muhammad Yusuf Insy

NIM : 2511532003

Dosen Pengampu : DR. Wahyudi,S.T,M.T.

Asisten Praktikum : jovan



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS TAHUN 2025

Daftar Isi

BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	3
BAB II.....	4
PEMBAHASAN.....	4
2.1 Pengertian Doubly Linked List.....	4
2.2 Cara Kerja Program	4
2.3 Analisis Kode Program	4
BAB III	10
PENUTUP.....	10
3.1 Kesimpulan	10
DAFTAR PUSTAKA.....	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman karena digunakan untuk mengatur dan menyimpan data agar dapat diakses secara efisien. Salah satu struktur data yang sering digunakan adalah Linked List. Linked List terdiri dari beberapa node yang saling terhubung menggunakan pointer.

Doubly Linked List (DLL) adalah pengembangan dari Singly Linked List yang memiliki dua pointer pada setiap node, yaitu pointer next untuk menuju node berikutnya dan pointer prev untuk menuju node sebelumnya. Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan dari depan maupun dari belakang.

Pada praktikum ini dibuat beberapa program Doubly Linked List menggunakan bahasa pemrograman Java, yaitu program pembuatan node, penambahan node, penghapusan node, serta penelusuran node. Praktikum ini bertujuan agar mahasiswa memahami konsep dasar DLL dan implementasinya dalam program Java.

1.2 Tujuan

1. Memahami konsep Doubly Linked List.
2. Memahami cara membuat node pada DLL.
3. Memahami proses penambahan node pada DLL.
4. Memahami proses penghapusan node pada DLL.
5. Memahami proses penelusuran maju dan mundur pada DLL.
6. Mengimplementasikan Doubly Linked List menggunakan Java.

1.3 Manfaat

1. Menambah pemahaman mengenai struktur data Linked List.
2. Melatih kemampuan logika pemrograman Java.
3. Memahami penggunaan pointer next dan prev.
4. Membantu mahasiswa memahami manipulasi data pada linked list.
5. Menjadi dasar dalam pembuatan struktur data yang lebih kompleks.

BAB II

PEMBAHASAN

2.1 Pengertian Doubly Linked List

Doubly Linked List adalah struktur data linear yang terdiri dari kumpulan node. Setiap node memiliki tiga bagian, yaitu:

- Data
- Pointer next (menuju node berikutnya)
- Pointer prev (menuju node sebelumnya)

Keunggulan Doubly Linked List dibandingkan Singly Linked List adalah data dapat ditelusuri dari depan ke belakang maupun dari belakang ke depan.

Contoh ilustrasi:

1 ⇌ 2 ⇌ 3 ⇌ 4 ⇌ 5

2.2 Cara Kerja Program

Program yang dibuat terdiri dari empat kelas utama:

1. NodeDLL_2511532003
Digunakan untuk membuat struktur node DLL.
2. InsertDLL_2511532003
Digunakan untuk menambahkan node di awal, akhir, dan posisi tertentu.
3. hapusDLL_2511532003
Digunakan untuk menghapus node di awal, akhir, dan posisi tertentu.
4. PenelusuranDLL
Digunakan untuk melakukan traversal maju dan traversal mundur.

2.3 Analisis Kode Program

A. Analisis Program NodeDLL_2511532003

`package pekan6_2511532003;`

Digunakan untuk menentukan package tempat class berada.

```
public class NodeDLL_2511532003 {
```

Mendeklarasikan class NodeDLL_2511532003.

```
int data_2003;
```

Variabel untuk menyimpan data pada node.

```
NodeDLL_2511532003 next_2003;
```

Pointer yang menunjuk node berikutnya.

```
NodeDLL_2511532003 prev_2003;
```

Pointer yang menunjuk node sebelumnya.

```
public NodeDLL_2511532003(int data_2003)
```

Constructor untuk membuat node baru.

```
this.data_2003 = data_2003;
```

Mengisi data node.

```
this.next_2003 = null;
```

```
this.prev_2003 = null;
```

Mengatur pointer next dan prev menjadi null karena node belum terhubung dengan node lain.

B. Analisis Program InsertDLL_2511532003

1. Method insertBegin()

Method ini digunakan untuk menambahkan node di awal DLL.

```
NodeDLL_2511532003 new_node_2003 =  
new NodeDLL_2511532003(data_2003);
```

Membuat node baru.

```
new_node_2003.next_2003 = head_2003;
```

Node baru menunjuk ke head lama.

```
head_2003.prev_2003 = new_node_2003;
```

Head lama menunjuk kembali ke node baru.

```
return new_node_2003;
```

Node baru dijadikan head baru.

2. Method insertEnd()

Method ini digunakan untuk menambahkan node di akhir DLL.

```
while (curr_2003.next_2003 != null)
```

Traversal sampai node terakhir.

```
curr_2003.next_2003 = newNode_2003;
```

Menghubungkan node terakhir dengan node baru.

```
newNode_2003.prev_2003 = curr_2003;
```

Node baru menunjuk ke node sebelumnya.

3. Method insertAtPosition()

Method ini digunakan untuk menambahkan node pada posisi tertentu.

```
for (int i = 1; i < pos_2003 - 1  
&& curr_2003 != null; ++i)
```

Traversal menuju posisi sebelum node yang akan disisipkan.

```
new_node.prev_2003 = curr_2003;
```

Node baru menunjuk node sebelumnya.

```
new_node.next_2003 = curr_2003.next_2003;
```

Node baru menunjuk node berikutnya.

```
curr_2003.next_2003 = new_node;
```

Node sebelumnya menunjuk node baru.

4. Method printList()

Digunakan untuk mencetak seluruh isi DLL.

```
while (curr_2003 != null)
```

Traversal dari head sampai akhir list.

C. Analisis Program hapusDLL_2511532003

1. Method delHead()

Digunakan untuk menghapus node pertama.

```
head_2003 = head_2003.next_2003;
```

Head dipindahkan ke node berikutnya.

```
head_2003.prev_2003 = null;
```

Pointer prev pada head baru dihapus.

2. Method delLast()

Digunakan untuk menghapus node terakhir.

```
while (curr_2003.next_2003 != null)
```

Traversal menuju node terakhir.

```
curr_2003.prev_2003.next_2003 = null;
```

Node sebelum node terakhir diputus dari node terakhir.

3. Method delPos()

Digunakan untuk menghapus node pada posisi tertentu.

```
curr_2003.prev_2003.next_2003 =  
curr_2003.next_2003;
```

Node sebelumnya dihubungkan langsung ke node setelahnya.

```
head_2003 = curr_2003.next_2003;
```

Jika node yang dihapus adalah head maka head dipindahkan.

D. Analisis Program PenelusuranDLL

1. Method forwardTraversal()

Digunakan untuk menelusuri DLL dari depan ke belakang.

```
curr_2003 = curr_2003.next_2003;
```

Berpindah ke node berikutnya.

2. Method backwardTraversal()

Digunakan untuk menelusuri DLL dari belakang ke depan.

```
curr_2003 = curr_2003.prev_2003;
```

Berpindah ke node sebelumnya.

3. Fungsi Main

Pada method main dibuat DLL:

1 $\rightleftharpoons$ 2 $\rightleftharpoons$ 3

Kemudian dilakukan:

- Traversal maju
- Traversal mundur

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan dapat disimpulkan bahwa:

1. Doubly Linked List merupakan struktur data yang memiliki pointer next dan prev.
2. DLL memungkinkan traversal dua arah, yaitu maju dan mundur.
3. Penambahan node dapat dilakukan di awal, akhir, dan posisi tertentu.
4. Penghapusan node dapat dilakukan di awal, akhir, dan posisi tertentu.
5. Bahasa Java dapat digunakan untuk mengimplementasikan struktur data DLL dengan baik.

DAFTAR PUSTAKA

- Abdul Kadir. 2018. *Dasar Pemrograman Java*. Yogyakarta: Andi Offset.
- Rinaldi Munir. 2016. *Algoritma dan Pemrograman*. Bandung: Informatika.
- Rosa A.S dan M. Shalahuddin. 2018. *Rekayasa Perangkat Lunak*. Bandung: Informatika.
- Oracle Corporation. "Java Documentation".